

操作系统结构

小背景：瓦萨沉船

1626 年到 1628 年间，瑞典国王古斯塔夫二世为了加强在波罗的海的海军力量，下令建造一艘续航能力强、容量大、火力猛、防护能力强的军舰。然而，这些极致的要求导致这艘军舰被建成重心极高、笨重且不符合物理规律的畸形船只。由于这些设计缺陷的存在，这艘军舰在出海航行不到 1 500 米，就被一阵微风吹倒倾覆。在沉没了 300 多年后的 1961 年 4 月 24 日，瓦萨号被几近完整地打捞上岸，当前展出在瑞典斯德哥尔摩的瓦萨沉船博物馆，全世界已有几千万游客参观了这艘具有不合理结构但又具有较强历史意义的军舰。

复杂系统的不同需求之间往往存在矛盾，在构建复杂系统时，必须合理考虑其内部结构，在不同的需求之间进行权衡。相比瓦萨号，现代操作系统的结构要复杂得多。如表 2.1 所示^①，最早的 UNIX V6 仅有 1 万行左右代码（不含驱动代码），Linux 0.01 版只有 8 102 行代码，而 Linux 5.7 内核则达到了 2 870 万行代码，且以每年约 200 万行代码的数量在快速变化。对操作系统这样复杂的系统而言，合理的架构与设计是不可或缺的，否则就可能重蹈瓦萨号的覆辙。

表 2.1 典型操作系统的代码规模

操作系统	代码行数
UNIX V6	1 万行
Linux 0.01	8 102 行
Linux 5.7	2 870 万行
Windows XP	4 500 万行
Windows 8	6 000 万行

操作系统的设计需要满足一定的目标，这些目标主要可以分为两大类：

- 用户目标：方便使用、容易学习、可靠、安全、流畅等。
- 系统目标：易于实现与维护、灵活、可靠、不易出错、高效等。

通常，这些目标之间存在共性，但也不可避免地存在矛盾。如果像瓦萨号一样，仅

^① Windows 的代码规模是基于网络上获得的信息估算的。

仅偏向于满足用户目标而不充分考虑系统目标，那么操作系统很可能实现不出来，即使实现出来也很难正常运行。事实上，在操作系统发展的 60 多年历史中，曾多次出现因过于强调各种极致能力而导致设计结构不合理并最终失败的案例。例如，1991 年到 1995 年间，IBM 雄心勃勃地投入了 20 亿美元，试图打造 Workplace 操作系统。然而，由于目标过于宏伟，系统过于复杂，最终导致项目失败^[1]。历史上的例子还有苹果的 Copland 操作系统项目^①、微软的 Windows VISTA 操作系统^②等。

本章将主要围绕操作系统的设计方法和典型的操作系统结构展开介绍，从而帮助读者理解现代操作系统中的一些基础设计方法与架构。

2.1 操作系统的机制与策略

本节主要知识点

- ❑ 如何控制操作系统的复杂性？
- ❑ 什么是机制？什么是策略？为什么要将二者尽可能分离？
- ❑ 操作系统内核有哪些典型的架构？

如何有效地控制操作系统设计的复杂性呢？操作系统乃至计算机系统中控制复杂性的一个重要设计原则是将策略与机制相分离^[2]。其中，策略（Policy）表示“要做什么”，机制（Mechanism）表示“如何做到”。如表 2.2 所示，对于计算机登录认证系统而言，机制包括输入处理、策略文件管理、桌面启动加载等，策略则包括什么用户、以什么权限登录等。另一个例子是操作系统的调度系统，策略包括先到先服务（First-Come-First-Service）、时间片轮转（Round Robin）、最短截止时间优先（Earliest Deadline First）、完全公平调度（Completely Fair Scheduling）等，机制则包括调度队列的设计、调度实体的表示（如线程）与时钟的中断处理等。

表 2.2 操作系统中典型的策略与机制

	策略	机制
登录	什么用户、以什么权限登录等	输入处理、策略文件管理、桌面启动加载等
调度	先到先服务、时间片轮转等	调度队列的设计、调度实体的表示、时钟的中断处理等

通过分离机制与策略，操作系统一方面可以通过设置多种不同的策略来适应不同的应用需求，而不需要重新实现对应的具体机制；另一方面也可以通过持续优化具体的机制来不断完善对具体策略的支持。

① 参见 [https://en.wikipedia.org/wiki/Copland_\(operating_system\)](https://en.wikipedia.org/wiki/Copland_(operating_system))。
② 参见 <https://medium.com/@benbob/what-really-happened-with-vista-an-insiders-retrospective-f713ee77c239>。

2.2 操作系统复杂性的管理方法

本节主要知识点

- 什么是 M.A.L.H 方法？
- 分层与层级有哪些区别？

管理系统复杂性的重要方法是 M.A.L.H 方法^[3]，即模块化（Modularity）、抽象（Abstraction）、分层（Layering）和层级（Hierarchy）。

模块化。模块化是通过分而治之（Divide and Conquer）原则，将一个复杂系统分解为一系列通过明确定义的接口进行交互的模块，并严格保障模块之间的界限。模块划分并不是越细越好，过多的模块反而会因为模块之间联系过多而无益于复杂性的控制；划分要充分考虑高内聚和低耦合，使模块具有较好的独立性。现代操作系统都存在一定程度的模块化结构，包括进程管理、内存管理、网络协议栈、设备驱动等。

抽象。抽象是在模块化的基础上将接口与内部实现分离，从而使模块之间只需通过抽象的接口进行相互调用，而无须关心各个模块间的内部实现。设计良好的抽象应该尽可能依从模块间的自然边界，并尽可能减少模块间的交互，从而减少错误在模块间的传递，提高整体系统的开发效率、质量与性能等。例如，UNIX 系列操作系统所提供的虚拟内存为物理内存提供了良好的抽象，使应用程序无须关心物理地址的具体位置，而只需要针对独立的、连续的虚拟地址空间进行设计。同样，文件系统抽象使得应用程序无须关心数据在物理介质（如闪存或磁盘）中的具体位置，而只需要通过定义好的文件系统接口（如 open、read、write 等）操作相应的数据。

小知识：宽进严出原则

模块的接口应该容忍各种可能的输入，抑制错误甚至恶意的输入，避免错误或恶意输入的效果在模块内传播，并且尽可能严格地控制模块对外的输出，从而减少错误在模块间的传播。

良好的模块化与抽象可以很好地将一个大系统分解为一系列交互较好的模块。然而，对于一个大型复杂系统而言，一个可能的结果是分解的模块非常多，相互的交互关系非常复杂（如图 2.1 所示），因此需要进一步通过分层与层级来控制复杂性。

分层。分层是通过将模块按照一定的原则进行层次划分，约束每层内模块间与跨层次模块间的交互方式，从而有效地减少模块之间的交互。通常的原则是：一个模块只能和同层模块以及相邻的上层或下层模块进行交互，而不能跨层和再上一层或再下一层的模块进行交互。分层也是构建复杂系统的一个重要方式：确定层级后，我们可以先构建